



INDIA.RUNS

Build what next India runs on

 TEAM OVERVIEW

Alpha Decoder

Team Details

Team Leader: Ayush

Problem Statement

Develop an intelligent candidate discovery and ranking system capable of identifying the most relevant candidates for an AI/ML hiring requirement from a dataset of **100,000** candidate profiles.



The solution should evaluate candidate suitability using experience, skills, career evidence, and behavioral signals while providing transparent and explainable ranking decisions.

Solution Overview

Proposed Solution

Our proposed solution is an intelligent candidate discovery and ranking system designed to identify the most relevant candidates from a dataset of **100,000 profiles**.

- Evaluates professional experience and role relevance
- Assesses technical skills, career achievements, and AI/ML expertise
- Incorporates behavioral hiring signals and recruiter-readiness indicators
- Produces a ranked shortlist with clear, explainable reasons

What Sets It Apart

Traditional matching systems rely heavily on keyword-based filtering. Our approach goes further by assessing context, experience quality, and practical expertise.

- Analyzes title relevance and career evidence
- Weights AI-specific skills and hiring readiness signals
- Uses indicators such as recruiter response rate, GitHub activity, notice period, and open-to-work status
- Provides an explainable reasoning layer for every recommendation

JD Understanding & Candidate Evaluation

Key Requirements from the JD

- Search & retrieval expertise
- Ranking & recommendation systems
- AI/ML engineering skills
- **Python, NLP & vector databases**
- **5–8 years** relevant experience

Most Important Candidate Signals

- Title & experience match
- AI/ML skills alignment
- Project & career evidence
- Open-to-work & hiring signals
- Explainable multi-factor scoring



Evaluation approach: The solution prioritizes relevance beyond keywords by combining skills, experience, and hiring readiness into a transparent scoring framework.

Ranking Methodology

Retrieve and score

- Extract candidate profile data
- Generate candidate features
- Calculate multi-factor scores
- Rank candidates by final score
- Export top recommendations

Models and heuristics

- Rule-based scoring engine
 - Weighted ranking framework
 - Keyword and career evidence matching
 - Title relevance boosting
 - Behavioral signal scoring
-

Final ranking signals

- **Experience score**
- **Title relevance score**
- **AI/ML skill score**
- **Career evidence score**
- **Behavioral signal score**
- **Negative match penalty**

Explainability & Data Validation

Every ranking decision is traceable, explainable, and supported by profile evidence.

Explainability

- Score breakdown for every candidate
- Highlighted matching skills
- Career evidence displayed
- Ranking reason generated

Data Validation

- Uses only candidate profile data
- No generated facts
- Evidence-based scoring
- Transparent ranking logic

Quality Controls

- Missing data reduces score
- Negative signals apply penalties
- Irrelevant profiles filtered out
- Low-quality profiles ranked lower

Workflow Overview

End-to-End Workflow

From job description ingestion to explainable candidate ranking



1. Job Description Input



2. JD Requirement Extraction



3. Candidate Profile Processing



4. Feature Extraction



5. Multi-Factor Candidate Scoring



6. Ranking & Sorting



7. Explainable Reason Generation



8. Final Top Candidate Output



Processing scale:

100,000 candidate profiles are evaluated to produce an explainable ranked shortlist of the most relevant **AI/ML** candidates.

System Architecture

End-to-end pipeline from raw candidate data to ranked hiring recommendations

Input Layer

- Job description
- Candidate profiles

Processing Layer

- Text extraction
- Feature extraction
- Signal detection

Scoring Layer

- Experience score
- Title score
- Career evidence score
- AI skill score
- Behavior score

Output Layer

- Ranked Top **100** candidates
 - Explainable reasoning
 - Submission CSV
-



Result: The architecture converts raw candidate data into explainable, ranked hiring recommendations.

RESULTS

PERFORMANCE

Results & Performance

Ranking Quality

What results or insights demonstrate ranking quality?

- Processed **100,000 candidate profiles**
- Identified **953 high-relevance AI/ML candidates**
- Top ranks were dominated by **Search Engineers, ML Engineers, and Recommendation Systems Engineers**
- Rankings aligned strongly with JD requirements and career evidence
- Every recommendation included **explainable ranking reasons**

Runtime & Compute Efficiency

How does the solution meet the challenge's runtime and compute constraints?

- Lightweight rule-based scoring engine
- Efficient single-pass candidate processing
- No large model inference required
- Low memory and compute usage
- Scalable to **100K+ profiles**

Key Metrics

100,000

Profiles Processed

953

Relevant Candidates

140.81

Highest Score

Validated

Submission

Successfully validated

TECHNOLOGY

STACK

Technologies Used

What technologies, frameworks, and tools were used – and why were they selected for this solution?

Technology	Purpose
Python	Core development and candidate ranking logic
JSON / JSONL	Processing large-scale candidate profile data
Rule-Based Scoring Engine	Transparent and explainable ranking decisions
Pandas	Data handling and analysis
CSV Export	Submission file generation
Git & GitHub	Version control and project management
VS Code	Development and debugging environment

Why These Technologies?

- **Python** enabled fast development and efficient data processing.
- **Rule-Based Scoring** ensured explainability and transparency.
- **Pandas** simplified large-scale data analysis.
- **GitHub** supported version tracking and reproducibility.

Key Design Principle

Simple • Explainable • Scalable • Compute Efficient

Submission Assets

A complete set of materials included for submission, review, and demonstration.

Included Deliverables

- **GitHub repository** with source code and project documentation
- **Demonstration video** showcasing the solution and workflow
- **Supporting files** for submission and review

What Each Asset Shows

- **Repository:** implementation details, version history, and reproducibility
- **Video:** end-to-end demonstration of the system in action
- **Files:** Presentation PDF, validated submission CSV, and supporting project documents

Repo URL: https://github.com/Ayush-eng-ai/Data_Analysis_Projects/tree/main/Hackathon_Projects/Redrob-Hackathon



INDIA.RUNS

Build what next India runs on

THANK YOU